

# AUTO SUGGESTION SYSTEM USING TRIE DATA STRUCTURE

## Mini Project Report

### Students Names

S.Yogasaikumar,M.Chethan Kumar Reddy,A.Cypriyan

### Department

Computer Science and Engineering(Data Science)

### Academic Year

2023-2027

---

## ABSTRACT

Auto Suggestion Systems are widely used in search engines, e-commerce websites, mobile keyboards, and dictionary applications to provide users with real-time suggestions while typing. The efficiency of such systems depends on the speed of searching and retrieving matching words.

This project implements an Auto Suggestion System using the Trie Data Structure. A Trie is a tree-based data structure that stores words efficiently and enables fast prefix-based searching. The system accepts a prefix from the user, searches the Trie, and displays all matching words. The project demonstrates the practical application of data structures in improving search performance and user experience.

---

## INTRODUCTION

With the growth of digital applications, users expect instant search results and suggestions. Traditional searching methods become inefficient when dealing with large datasets because every word must be checked individually.

The Trie Data Structure provides an efficient solution for prefix searching. Words are stored in a hierarchical structure where each node represents a character. This allows the system to quickly locate words that share a common prefix.

The Auto Suggestion System developed in this project uses a Trie to store words and generate suggestions based on user input.

---

# PROBLEM STATEMENT

Traditional searching techniques require scanning multiple words to find matches, which becomes time-consuming for large datasets.

The objective of this project is to develop an Auto Suggestion System that:

- Stores words efficiently.
- Performs fast prefix-based searches.
- Generates suggestions instantly.
- Improves user experience.

---

# OBJECTIVES

1. To understand the Trie Data Structure.
2. To implement word insertion into a Trie.
3. To perform efficient prefix searching.
4. To generate matching suggestions.
5. To demonstrate the practical use of data structures.

---

# METHODOLOGY

The project follows the following steps:

## Step 1: Create Trie Structure

A Trie consists of nodes where each node stores a character and links to child nodes.

## Step 2: Insert Words

Words are inserted character by character into the Trie.

Example Words:

- app
- apple
- application
- apply
- banana
- bat
- ball

### Step 3: Accept User Input

The user enters a prefix.

Example:

app

### Step 4: Search Prefix

The Trie is traversed according to the entered prefix.

### Step 5: Generate Suggestions

All words that start with the prefix are collected and displayed.

---

## FLOWCHART

Start



Initialize Trie



Insert Dictionary Words



User Enters Prefix



Search Prefix in Trie



Prefix Found?

→ No → Display "No Suggestions Found"

→ Yes → Traverse Subtree

↓

Collect Matching Words

↓

Display Suggestions

↓

End

---

## ALGORITHM

### Insert Operation

1. Start from the root node.
2. Read each character of the word.
3. Create child nodes if necessary.
4. Move to the next node.
5. Mark the last node as the end of the word.

### Suggestion Operation

1. Read the prefix.
  2. Traverse the Trie according to the prefix.
  3. If the prefix exists:
    4. Explore all child nodes.
    5. Collect matching words.
    6. Display suggestions.
  7. If the prefix does not exist:
    8. Display "No Suggestions Found".
- 

## IMPLEMENTATION

The project is implemented using Python and Object-Oriented Programming (OOP).

Main Components:

## **TrieNode Class**

Stores:

- Child nodes
- End-of-word indicator

## **Trie Class**

Provides:

- Word insertion
  - Prefix searching
  - Suggestion generation
- 

# **SAMPLE INPUT AND OUTPUT**

## **Input**

app

## **Output**

Suggestions:

1. app
2. apple
3. application
4. apply

## **Input**

ba

## **Output**

Suggestions:

1. banana
2. bat
3. ball

## **Input**

xyz

## Output

No Suggestions Found

---

## ADVANTAGES

- Fast searching
  - Efficient prefix matching
  - Scalable for large datasets
  - Real-time suggestions
  - Easy implementation
- 

## APPLICATIONS

- Search Engines
  - Mobile Keyboards
  - Dictionary Applications
  - Contact Search Systems
  - E-Commerce Platforms
  - Text Editors
- 

## FUTURE ENHANCEMENTS

- Frequency-based suggestion ranking
  - Spell correction
  - Voice search support
  - Database integration
  - AI-powered recommendations
- 

## CONCLUSION

The Auto Suggestion System using Trie Data Structure successfully demonstrates efficient prefix-based searching. By organizing words in a Trie, the system can quickly generate suggestions and improve user experience. This project highlights the importance of data structures in solving real-world problems and provides a strong foundation for advanced search and recommendation systems.

---

# REFERENCES

1. Python Documentation
2. Data Structures and Algorithms Textbooks
3. Trie Data Structure Concepts
4. Online Learning Resources and Tutorials